

Reviewer Pack — Minimal Order of Noncommutative K-theory and Resolution Proo...

Entry 95fe6257df14 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 19:17:40 UTC

Minimal Order of Noncommutative K-theory and Resolution Proof Width Correlation

Entry ID: 95fe6257df14 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 19:17:40 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry (K-theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For a given CNF formula φ , the minimal order of its associated noncommutative K-theory group $K(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $|K(\varphi)| = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Noncommutative K-theory provides a generalization of classical K-theory to the context of algebras, which may expose hidden structure in the complexity of resolution proofs. The noncommutative nature could lead to new insights into proof size and complexity.

Taxonomy category: KTHEORY_RESOLVER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 43b3c2d8e2766ea5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 24 out of 30 random seeds yield a correlation coefficient $r \geq 0.7$ between the minimal order of noncommutative K-theory group and resolution proof width, with all seed results within 5% deviation from the mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.80	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative geometry" AND "resolution proof complexity" AND K-theory" - "minimal order noncommutative K-theory" AND resolution proof width" - "correlation minimal order K-theory group" AND resolution proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            if random.choice([True, False]):
                clause[0], clause[1] = clause[1], clause[0]
            cnf.append(clause)
        return cnf

    def cnf_to_algebraic_structure(cnf):
        variables = set()
        for clause in cnf:
            for literal in clause:
                variables.add(abs(literal))
        n = max(variables)
        algebraic_structure = [0] * (2 * n + 1)
        for literal in range(-n, n + 1):
            if literal != 0:
                algebraic_structure[literal] += 1
        return algebraic_structure

    def resolution_width(cnf):
        queue = cnf[:]
        while True:
            new_clause = None
            for i in range(len(queue)):
                for j in range(i + 1, len(queue)):
                    new_clause = None
```

```

        if set(queue[i]).isdisjoint(set(queue[j])):
            continue
        common_vars = [var for var in queue[i] if -var in queue[j]]
        if common_vars:
            new_clause = [-common_vars[0]] + [v for v in queue[i] if v != -common_vars[0]]
            break
    if new_clause:
        break
    if not new_clause:
        return len(queue)
    queue.append(new_clause)

n = random.randint(5, 40)
cnf = generate_cnf(n)
algebraic_structure = cnf_to_algebraic_structure(cnf)
resolution_width_value = resolution_width(cnf)

metric_name = "resolution_width"
metric_value = resolution_width_value
instances_tested = 1
n_max = n
conjecture_holds = False
counterexample = ""

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if abs(result["metric_value"] - mean_value) <= std_value)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the expected time frame. | next: Re-run the test with a longer timeout or increase the computational resources to ensure it can complete and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12665
2	preregistration	ollama_remote	glm4:latest	0	0	9265
3	novelty	ollama_remote	glm4:latest	0	0	8394
4	novelty	ollama_remote	glm4:latest	0	0	8880
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12157
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7873
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12013
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10911
9	judge	ollama_remote	glm4:latest	0	0	11667

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 93824 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/95fe6257df14.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/95fe6257df14.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/95fe6257df14.tar.gz` (if generated)